Grocery Placement Robot

Zhenkai Gao, Alex Kautz, Pascale Leone

Abstract

This project explores the task of putting away groceries with a robotic arm, addressing the computer vision and motion planning components and their translation into a simulated environment. We then narrow our focus to the subtask of picking up objects, analyzing the effect of lighting on the YOLO detection algorithm and the effect of increasing movement tolerance on the success of the pickup task[1].

We find that lower position tolerance significantly improves grasp success, while upward shadow placement noticeably changes YOLO's ability to detect objects consistently.

1 Introduction

Imagine a future where you receive a box of groceries from a delivery service (like Amazon), take the box to your kitchen table, and open it. Then, a helpful robotic arm takes items out of the box and puts them away. The arm knows the best place for each item, be it the cupboard, fridge, or freezer.

This kind of assistance can be a powerful utility for elderly or disabled individuals, as well as for professional kitchens where inventory management is a major time sink.

For our project, we simulated a version of this arm using Gazebo in order to explore the problem space. We designed an environment analogous to a real kitchen, and laid out a design for the full system.

We then focused on just the sub-problem of picking up objects, integrating computer vision techniques like YOLO, and trajectory planning tools like MoveIt. We analyze the effect inverted images and shadows may have on the YOLO algorithm. We also measured the effect that lowering the position and orientation tolerance in trajectory planning has on the reliability of the picking up task.

We hope this work can lead to real-world implementations of the grocery placing system, and broadly help with picking tasks for robotic arms.

2 Background and Related Work

Much research has been done in the field of home service robotics. For example, assisting the elderly with household tasks is a common application for robotics. In 1998, a team based in Italy developed MOVAID. The MOVAID system consisted of a robotic arm attached to a driving base.

[1] Our code can be found at https://github.com/AlexKautz/GroceryRobot

MOVAID's goal was to assist in 3 main tasks: heating up food, cleaning the kitchen, and changing bed sheets. As part of that first task, the robot had to take food out of the refrigerator [1].

Researchers at the School of Artificial Intelligence, Jingchu University of Technology present a novel framework for an in-home service robot [13]. They utilize ROS to develop high-precision robotic manipulators for enhanced grasping capabilities. They evaluate the success of their robot in a simulated home environment. In addition, the authors employ vision model YOLOv5 for object detection and recognition.

Similar to these works, we want to use a robotic arm to unpack and place groceries. We believe this task is important because it can help the elderly or disabled in the kitchen. We use a static robot arm rather than a moving base and focus on the task of putting away groceries. This task involves three key areas: object detection, trajectory planning, and stable placement.

Object detection in a cluttered environment is an important problem. Researchers at Jaume I University in Castellon de la Plana, Spain studied object detection in a cluttered scene in order to help the BAXTER robot pick the correct object in a cluttered environment [7]. They employed deep learning methods to improve performance, and compare using ResNet, a semantic segmentation based method, to a localization method with YOLO for object detection.

Trajectory planning is also relevant to our problem. Our robotic arm must navigate from the box of groceries to the appropriate storage container, without hitting any obstacles. Researchers in the Department of Mechatronics Engineering at the Isparta University of Applied Sciences in Isparta, Turkey studied trajectory planning for a robotic arm with 6 degrees of freedom [14]. Their experiments focused on the Particle Swarm Optimization (PSO) algorithm, which is a population-based stochastic optimization method. This algorithm starts with a population of random solutions and searches for the optimal solution by updating generations. They proposed that the PSO algorithm optimizes trajectory planning of the 6-degree-of-freedom arm.

It is also important that our robot is able to facilitate stable placement of each grocery item. As the storage containers accumulate objects, the robot must determine an appropriate location for the new grocery item. Haustein, Hang, Stork, and Kragic address the problem of placement planning by creating an algorithm that computes placements for objects based on three main principles: (1) suitable locations must be identified, (2) placements must be reachable, and (3) not all placements are equally desirable [2].

Recent work in robotic manipulation has shifted from traditional model- and planning-based approaches toward data-driven and learning-based methods. Kleeberger et al. survey this transition, emphasizing the rise of learning-based grasping and its challenges in generalization [5]. While our project focuses on a single robotic arm, real-world home service robots require whole-body coordination, including mobile bases and multiple manipulators. Jiang et al. propose a benchmark and system framework for real-world household tasks [3].

In addition, recent approaches explore unified learning frameworks that jointly model perception, navigation, and interaction, enabling robots to actively improve performance through interaction with the environment [12]. Finally, vision-language models allow robots to perform more semantic, task-aware manipulation, such as grasping specific object parts and generalizing to unseen objects [9].

3 Methods

3.1 Theoretical Framework

3.1.1 Computer Vision

To detect, recognize, and locate our object we use a YOLO (You Only Look Once) model. YOLO is an object detection and segmentation model initially developed by Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi at the University of Washington in 2015 [11]. Their model differs from prior object detection techniques because they frame object detection as a regression problem. A single convolutional neural network simultaneously predicts bounding boxes and class probabilities. This allows for incredibly fast and generalizable object detection.

In our project, we use the COCO-pretrained YOLOv8 model, released in 2023 by Ultralytics [4]. COCO is a large-scale annotated object detection and segmentation dataset. The classes in this dataset are diverse, containing categories such as cars, animals, or handbags. This dataset is appropriate for the scale of our problem, as it includes several grocery items as classes, specifically apple, orange, banana, carrot, and broccoli. In our scaled-down simulation we are using a red ball to represent an apple. The COCO dataset has a category called 'sports ball'. Since we are using a single fixed item, COCO is an adequate dataset for our object detection task.

Our overhead camera uses YOLOv8 to detect the ball's bounding box. From there it computes and stores the centroid of the ball. We then compute the depth level based on the centroid coordinates. With these x,y,z values we transform the pixel location to the location in the simulation world. This information is published for use in the trajectory calculation.

3.1.2 Trajectory Motion Planning

The trajectory motion planning module connects the overhead camera perception output to the UR3e arm motion execution through MoveIt. After the camera perception node detects the target grocery object, it publishes the object's 3D position as a PointStamped message. The motion planning node subscribes to this coordinate and generates a pickup sequence consisting of a pre-grasp pose, grasp pose, and lift pose. These poses were obtained by adding vertical offsets to the detected object position. Each target pose is converted into a MoveIt MoveGroup action goal with position and orientation constraints for the end-effector link. Then MoveIt plans and executes collision-aware trajectories for the UR3e manipulator to approach the object, reach the grasp pose, and lift the object after the gripper closes. The gripper is controlled separately through joint trajectory commands, allowing the system to coordinate arm motion and grasp execution in a simple closed-loop pickup pipeline.

3.2 Technical Approach

3.2.1 Simulation Environment

For our simulation, we employed Gazebo [6], an open-source robotics simulator with deep integration into ROS [10]. Gazebo enables the construction of digital replicas of physical robots within customizable 3D environments.

We designed an environment that was as simple as possible while still representing our project. We used a two-tier shelf to represent the cabinet, refrigerator, and freezer combination. This retained

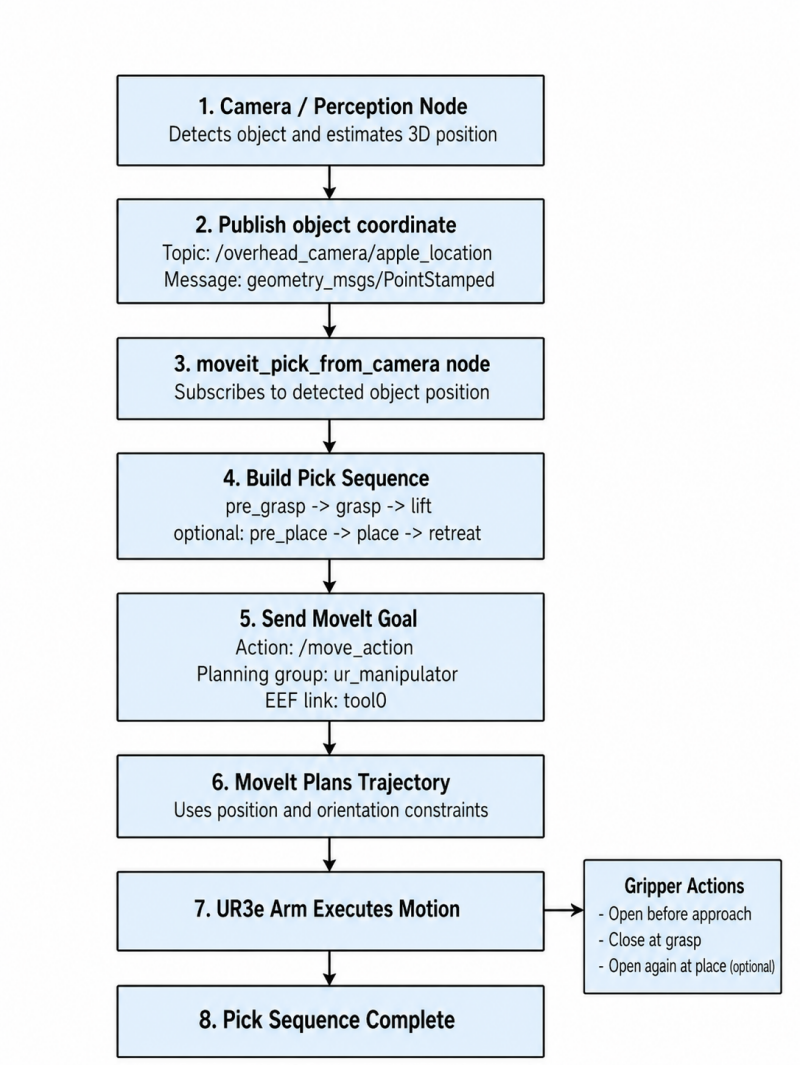


Figure 1: Pipeline of the MoveIt object pickup node

the concept of having different locations to place objects. We used a simple surface opposite the robot in place of the table and box that contain the groceries. Finally, for the groceries themselves, we used basic shapes, such as a sphere for the apple. You can see the environment running inside Gazebo in Figure 2.

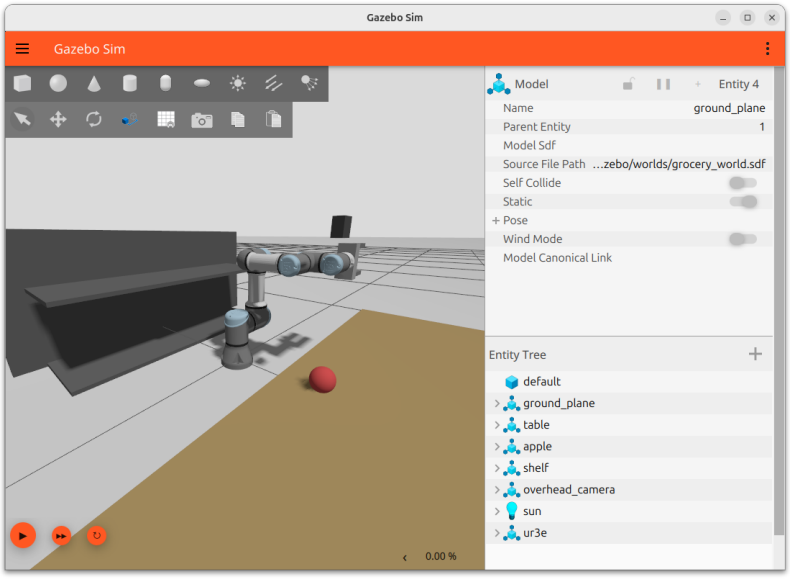


Figure 2: Pictured in the environment: (1) two tier shelf, left; (2) robotic arm, center; (3) overhead camera, center; (4) table, bottom right; (5) apple, bottom.

The environment also contains an overhead camera that sends images to the ROS system, where they are picked up by the computer vision nodes. This view only contains the apple, and can be seen in Figure 3.

Conceptually, the robot is treated as separate from the environment so that it can be swapped out, but it is positioned at the center of the scene, where it can reach both the groceries and the shelf while still facing a non-trivial placement task.

3.2.2 Robot Arm and Gripper

For this project, we selected the UR3e robotic arm from Universal Robots as the main manipulator in our simulated environment. The UR3e is a compact 6-DoF collaborative robot arm that is commonly used in industrial and research settings. Its collaborative design makes it suitable for applications involving human environments, which aligns with our goal of developing a household grocery assistance robot.

The UR3e was also chosen for its compatibility with our software stack. Since our project uses Gazebo, ROS 2 Kilted, and MoveIt, the existing Universal Robots ROS 2 packages were useful because they provide robot description files, driver support, and MoveIt configuration resources. This made it easier to spawn the UR3e in Gazebo and integrate it into our motion planning pipeline.

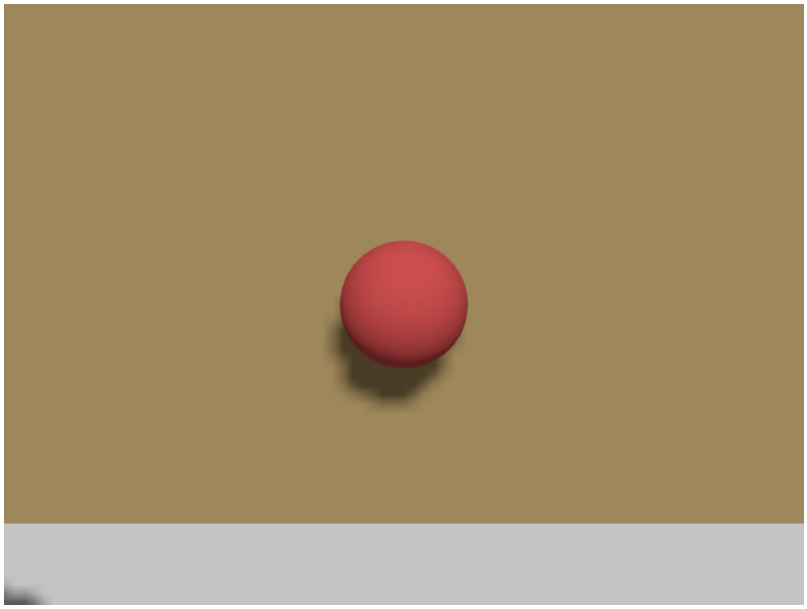


Figure 3: View of the environment from the overhead camera.

The arm's six degrees of freedom allow the end effector to move and orient itself in 3D space, which is necessary to manipulate grocery objects. However, the UR3e has a relatively limited workspace compared with larger arms such as the UR5 or Kinova Gen3. Therefore, the table, shelf, and grocery objects had to be placed within the arm's reachable range. Some target poses may also fail due to inverse kinematics constraints, joint limits, or unfavorable orientations. These cases are further discussed in the experimental results section.

For the gripper, we used a simplified two-block design in simulation. The two blocks open and close to represent a basic grasping mechanism. Although this design is not intended to be a physically realistic final gripper, it was sufficient for demonstrating the core grocery-pickup pipeline.

3.2.3 ROS

Robot Operating System 2, shortened as ROS 2, serves as the base of our software stack [10]. It is a robotics middleware that allows different parts of the system to communicate through topics, messages, services, and actions. In our project, we built several ROS 2 nodes that publish and subscribe to shared topics, allowing perception, localization, motion planning, and gripper control to work together as one coordinated system.

The ROS 2 nodes built for our project are:

`overhead_camera_localizer` Subscribes to the overhead RGB camera, depth camera, and camera information topics to detect the apple and estimate its 3D position. It publishes the apple location in the world frame as a `PointStamped` message, along with an annotated camera image and visualization marker.

`moveit_pick_from_camera` Subscribes to the apple position published by the overhead camera and builds a pick sequence including pre-grasp, grasp, and lift poses. It sends these pose goals to MoveIt through the action server and also publishes gripper commands during the grasping sequence. It is used during the tolerance experimentation.

`test_apple_detection` Tests the `overhead_camera_localizer` by moving the apple to 5 different locations in sequence. It then saves an image of the apple with an annotated bounding box from YOLO. It is used during the object detection experiment.

`joint_control_panel` Runs a GUI App with sliders to directly change the position of the robotic arm. Used during initial development for debugging.

3.3 Methodology

For our experiment design we decided to focus on just the "pick up" component of the overall grocery placement task. This allowed us to understand the effect changes to tolerance and lighting have to the process.

3.3.1 Tolerance Variation Experiment

To evaluate the effect different levels of tolerance have on the pickup component, we considered the variables `position_tolerance` and `orientation_tolerance`. These inform the ROS MoveIt package on how far from its goal position and orientation it can target when planning its kinematics [8]. We ran a large number of grasping attempts. For each attempt, we varied both the location of the ball and the tolerances. Five rows of our collected data are shown in Table 1.

Ball X (m)	**Ball Y (m)**	**Position Tolerance (m)**	**Orientation Tolerance (rad)**	**Result**
0.40	-0.10	0.01	0.02	SUCCESS
0.35	0.00	0.01	0.06	SUCCESS
0.40	0.10	0.02	0.05	SUCCESS
0.30	-0.10	0.05	0.02	MISSED
0.35	0.00	0.06	0.07	MISSED

Table 1: Sample of grasp trials showing ball position, tolerance parameters, and outcome.

We covered 8 possible values (from 0.01 to 0.08) each for position and orientation tolerance, along with 5 ball locations, totaling 320 grasp attempts. To collect this data efficiently, we used an automated script that ran Gazebo headless, re-spawning the ball and detecting whether the grasp succeeded.[2]

[2] A video of the robot running these trials can be found at https://vimeo.com/1190248547

3.3.2 Object Detection Experiment

To evaluate the YOLOv8 model's performance on object detection we conducted an experiment where we inverted the images received from the camera stream and varied the placement of the ball. Using inverted images resulted in different locations for the shadow relative to the ball: in the original image the shadow is below the ball, while in the inverted one it is above. We compared 5 locations for the ball in the camera frame.

4 Experimental Results / Technical Demonstration

4.1 Tolerance Variation Results

To analyze our collective data on tolerance, we added up the number of successful grasp attempts (out of 5) for each combination of position and orientation tolerance.

We expect that as both position tolerance and orientation tolerance increase, the number of successful grasps also decreases.

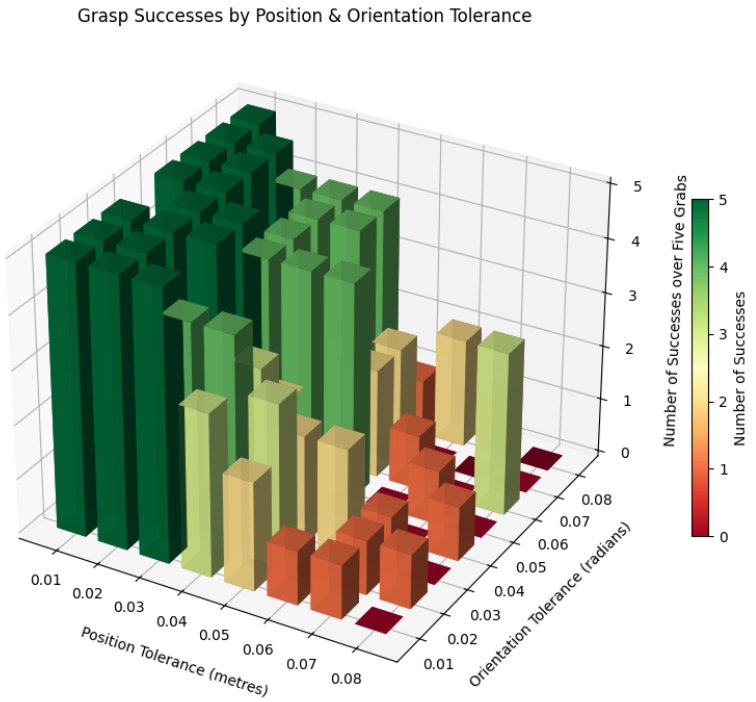


Figure 4: Bar chart of the number of successful grasps out of five attempts for each combination of position and orientation tolerance

Figure 4 clearly shows that our expectation on the effect of increasing position tolerance was correct; the number of successful pickup attempts drops continuously. We also see randomness in the data;

this is because a pickup attempt with a high tolerance can still get "lucky" and position itself correctly.

We do not see an effect of increasing the orientation tolerance within its range. This is most likely because the tested range was too narrow to observe an effect. Our range from 0.01 to 0.08 radians only covers around 4°. Future work should cover a larger range, of at least 30° (0.5 radians).

4.2 Object Detection Experiment Results

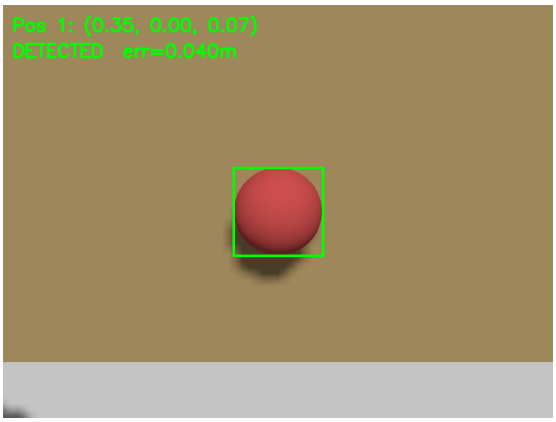
The results of our object detection experiment indicate that the YOLOv8 model does a better job of detecting the ball in the original image than in the inverted one. The model successfully detects the ball in all 5 trials given the original image, but is only able to detect the ball 2 times in the inverted image. Since model performance is largely dependent on the data it was trained on, we suppose that images of balls in the COCO dataset may be positioned such that the shadow appears below the ball. This test indicates that light source will affect our ability to detect objects. For our final program, we detect the ball using the original image, where the shadows are below the ball.

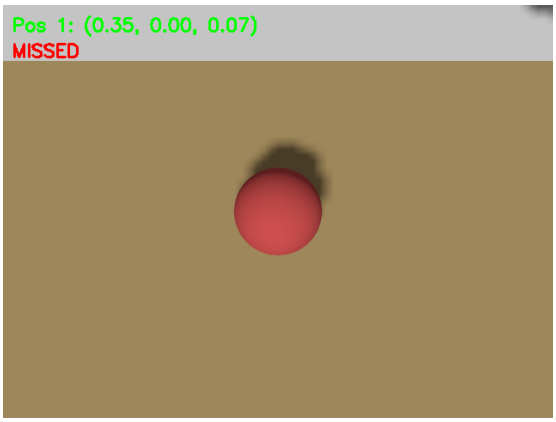
	Original	**Inverted**
True world position	Estimated world position	Estimated world position
(0.350, 0.000, 0.065)	(0.349, -0.001, 0.105)	NOT DETECTED
(0.300, 0.100, 0.065)	(0.305, 0.089, 0.116)	(0.297, 0.106, 0.040)
(0.300, -0.100, 0.065)	(0.305, -0.089, 0.117)	NOT DETECTED
(0.400, 0.100, 0.065)	(0.394, 0.089, 0.116)	(0.403, 0.106, 0.040)
(0.400, -0.100, 0.065)	(0.394, -0.089, 0.116)	NOT DETECTED

Table 2: True world position vs estimated world position for original and inverted images.

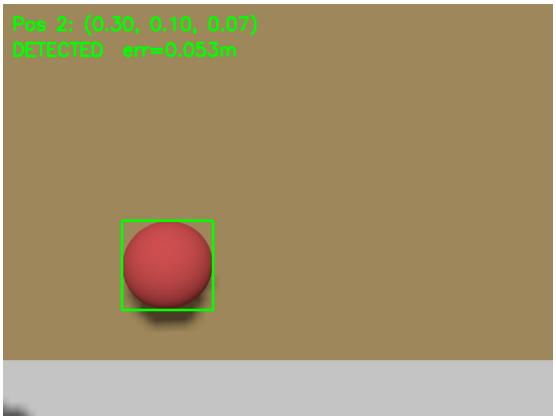
Visualization of Object Detection Bounding Boxes

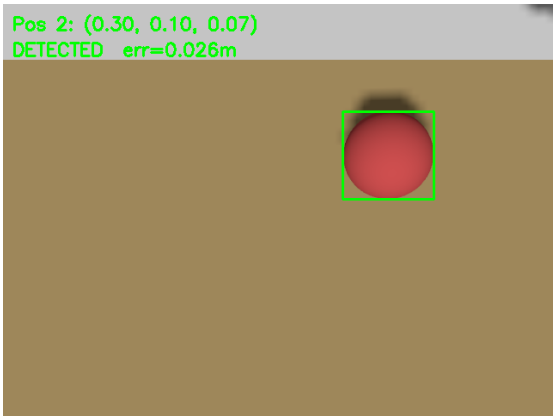
Location 1: **(0.350, 0.000, 0.065)**



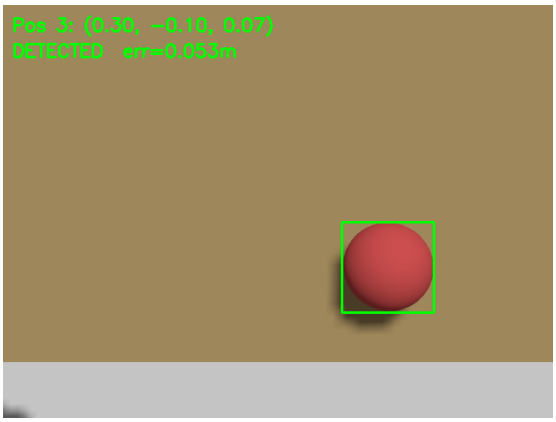


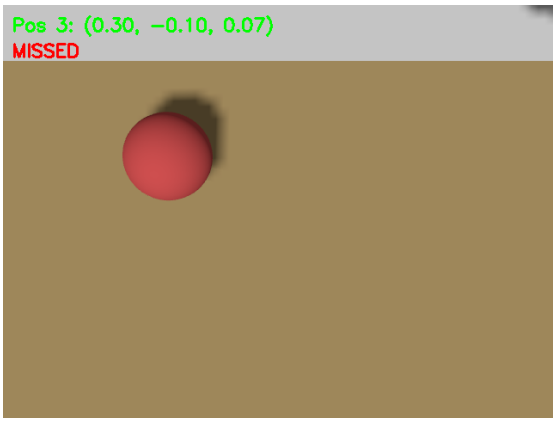
Location 2: **(0.300, 0.100, 0.065)**



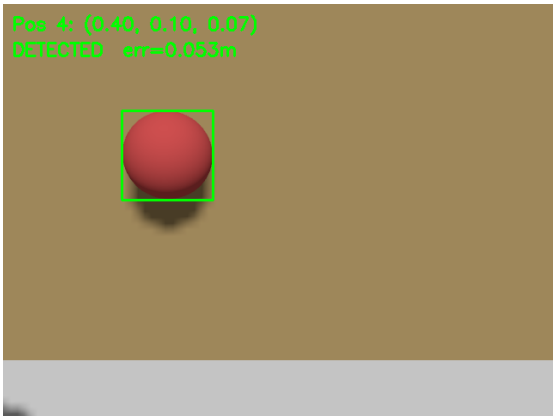


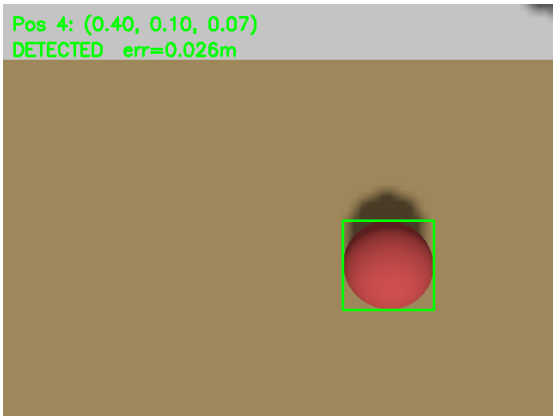
Location 3: **(0.300, -0.100, 0.065)**



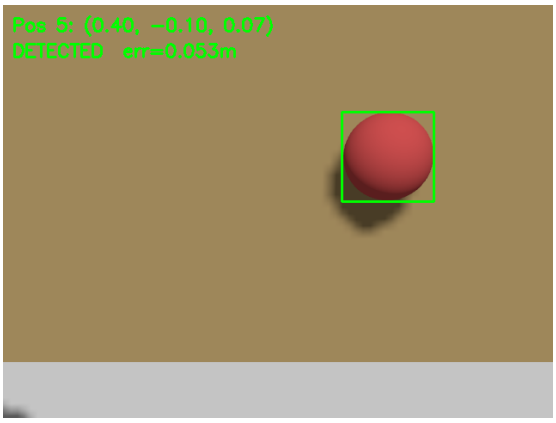


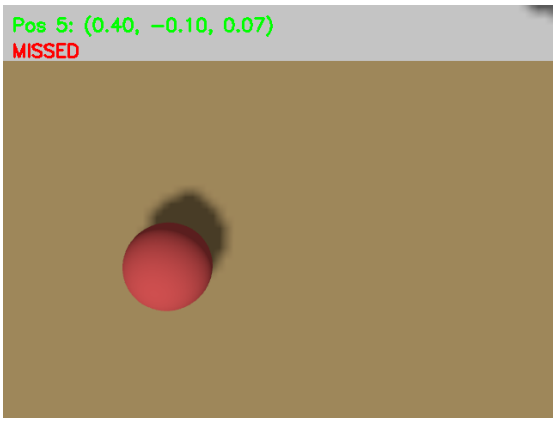
Location 4: **(0.400, 0.100, 0.065)**





Location 5: **(0.400, -0.100, 0.065)**





5 Conclusion and Future Work

We were able to successfully simulate a robotic arm in Gazebo and integrate YOLO and MoveIt for object detection and motion planning, respectively. Our results demonstrate that YOLOv8 pretrained on the COCO dataset is an appropriate vision model for our task, as we were able to successfully locate the ball and compute its centroid. In addition, we evaluated several conditions that affect the robot arm's performance. In our vision experiment, we tested how the location of shadows affects object detection performance. In our tolerance experiment, we measured the effect of the position and orientation tolerance on the robot arm's gripping abilities. These are all important factors we would need to consider if our Grocery Robot were to actually be deployed. Our final program for the robotic arm is able to identify the location of the ball, plan a trajectory to the ball, and pick it up using the gripper. This is an important stepping stone to building a successful grocery placement robot system.

There are many avenues for future work. Our initial proposal was to program a robotic arm in simulation to help a person unpack their grocery delivery order. We wanted our robot to classify grocery items as frozen, refrigerated, or non-perishable, and then place them in corresponding locations in the fridge or shelf. The next step would be to add motion planning to have our robotic arm place the ball in the correct location. Then, we would like to add additional simulated grocery objects, and see if the YOLO algorithm could successfully recognize different objects in a cluttered environment. Ultimately, we would like to deploy our robot in the real world. This would involve transferring the simulated Grocery Robot to the real UR3e arm.

Another interesting future work question would be to evaluate the position tolerance necessary to complete the task successfully. Our results show that lower tolerance has higher success rates. However since lower tolerance is more computationally expensive, we would like to find the highest possible position tolerance where the task can be completed.

References

[1] Paolo Dario, Eugenio Guglielmelli, Cecilia Laschi, and Giancarlo Teti. Movaid: a personal robot in everyday life of disabled and elderly people. *Technology and Disability*, 10(2):77–93, 1999.

[2] Joshua A. Haustein, Kaiyu Hang, Johannes Stork, and Danica Kragic. Object placement planning and optimization for robot manipulators. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7417–7424, 2019.

[3] Yunchu Jiang, Ruohan Zhang, Josiah Wong, Chen Wang, Yanjie Ze, He Yin, Cem Gokmen, Shuran Song, Jiajun Wu, and Li Fei-Fei. Behavior robot suite: Streamlining real-world whole-body manipulation for everyday household activities. In *Proceedings of the 9th Conference on Robot Learning (CoRL)*, Seoul, South Korea, 2025.

[4] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023.

[5] Kai Kleeberger, Richard Bormann, Werner Kraus, and Marco F. Huber. A survey on learning-based robotic grasping. *Current Robotics Reports*, 1:239–249, 2020.

[6] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, Sendai, Japan, 2004.

[7] Arijit Mallick, Angel P. del Pobil, and Enric Cervera. Deep learning based object recognition for robot picking task. In *Proceedings of the 12th International Conference on Ubiquitous Information Management and Communication*, IMCOM '18, New York, NY, USA, 2018. Association for Computing Machinery.

[8] MoveIt Contributors. MoveGroupInterface Class Reference, 2024. Accessed: 2025-05-07.

[9] Thomas Oort, Daniel Miller, Will N. Browne, Nicolas Marticorena, Jonathan Haviland, and Niko Suenderhauf. Open-vocabulary part-based grasping. *arXiv preprint arXiv:2406.05951*, 2025.

[10] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Ng. ROS: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, volume 3, 2009.

[11] Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. *CoRR*, abs/1506.02640, 2015.

[12] Shubham Uppal, Ankush Agarwal, Hao Xiong, Kyle Shaw, and Deepak Pathak. Spin: Simultaneous perception, interaction and navigation. *arXiv preprint arXiv:2405.07991*, 2024.

[13] Mengdan Yu, Xue Chen, Gehua Feng, and Hongfeng Zhu. Developing a new generation of home service robot based on ros. In *2025 44th Chinese Control Conference (CCC)*, pages 4834–4839, 2025.

[14] Özge Ekrem and Bekir Aksoy. Trajectory planning for a 6-axis robotic arm with particle swarm optimization algorithm. *Engineering Applications of Artificial Intelligence*, 122:106099, 2023.